

Report of the first module of the project:

Partitioned Hash Join with Duplicates

Andrea Lepori

Università di Pisa

Parallel and distributed systems: paradigms and models

April 2, 2026

Contents

1	Brief introduction to the problem and tasks	2
2	Design choices for the mapping function and for P (and why)	2
3	Performance Insight (What I expect)	2
4	Workflow	3
5	Evidence of GCC auto-vectorization (used flags, what the compiler reported, short excerpt of the GCC report)	3
6	Brief explanation of my intrinsic strategy	4
7	Experiments and optimization attempts	5
8	Performance results obtained on node09 and discussion of factor limiting performances(compute vs memory vs overhead)	5
9	Cuda optional part: correctness verification and breackdown of time components	6
10	Comparison between different versions	7
11	Suggested experiments	8

1 Brief introduction to the problem and tasks

The core problem of this assignment is to implement a **partition mapping kernel**. We are given as input an array of N keys (`uint64_t`). In output we want to generate an array of the same length $\text{part_id}[i] \in [0, P)$. We make the simplifying assumption $P = 2^k$. This is part of a **partitioned hash join**, where data is split into independent chunks for parallel processing. This is an embarrassingly data parallel workload because the key idea is that we want to apply the very same operation to many elements. In this module we want to test: SIMD on CPU (auto + intrinsics), possibly GPU (SIMT).

In crude words SIMD = same instruction on multiple data and GPU SIMT = many threads executing same kernel. Essentially in this first module we take a simple loop, the so called hot loop and we try to push it to the hardware limits. In our case the hot loop is the one that implements the partition mapping.

In the **first task** we write a plain C++ baseline loop. We compile it without vectorization (scalar version) and with auto-vectorization and we show the GCC report.

In the **second task** the goal is to use AVX2 Intrinsics to implement manual SIMD using `_mm256_*`. We check that it provides the same output as the baseline using a checksum.

The **third task** involves the use of CUDA to provide a GPU version. We have to measure the kernel time separately from the transfer time as well as to provide the compact correctness check using the checksum.

2 Design choices for the mapping function and for P (and why)

The partitioning function is implemented using a hash-based approach to ensure uniform distribution of keys across partitions. A hash function derived from the MurmurHash3 finalizer (64-bit variant) [1] [2] is used due to its strong bit-mixing properties that ensure good distribution of the keys, its computational efficiency (fast, bitwise operation: only shifts, XOR, AND... although it has a multiplication by a constant that, we will see, acts as a bottleneck...), its deterministic behaviour and the fact that is SIMD/GPU-friendly (no branches, same operations for all data, no memory dependencies each element independent). Since the number of partitions is a power of two, the modulo operation is replaced with a bitwise AND (`& (P - 1)`), which significantly reduces the computational cost. This choice is particularly beneficial for SIMD and GPU execution, as it avoids expensive division operations and maintains a fully parallel execution model.

3 Performance Insight (What I expect)

The kernel is memory bound (most likely), the sequential access should give good cache behavior and the fact that there are no dependencies make it a perfect SIMD candidate. I try to keep the hot loop simple because auto-vectorization works best on simple loops. The expected SIMD speedup $\approx \text{vector width} \times \text{efficiency}$. Since the computation done by the hash function is simple and not heavy I expect the execution time of the kernel to be limited by the transfer time from memory. So the efficiency term

should not be very good. Furthermore, there are also some bottlenecks for the cpu part due to the parallelization of the multiplication of 64 bit integer values and the conversion from 64 to 32 bit integers value which lack of the native SIMD support in AVX2, so to parallelize it I use a combination of more instructions introducing an overhead. To remove this overhead, one can think to remove directly the multiplication operation, or to make an hash function that returns a 32 bit int so that there is that direct hardware support for the SIMD multiplication and packing operations but you lose a bit of the property of the hash function to guarantee a good distribution of the keys that maybe will be useful in future, so it is a trade-off.

4 Workflow

I start simple with a clean baseline scalar version that generates the input deterministically (fixed random seed), implements the mapping function and the correctness check. The correctness check is basically a checksum: it applies the hash function to all the keys and accumulates the sum. I check that the computed sum is consistent across all versions. Then I try the auto-vectorization by specifying the relative compiler option and show the GCC report to prove that the auto-vectorization works. Then I do manual vectorization using AVX2 intrinsics, specifically processing $4 \times \text{uint64}$ per iteration with the use of `__mm256_*`. After that I try to do some "optimization attempts" or better changes/experiments in the code to see if there are any improvements, I briefly describe them and their logic in the relative section.

Finally I implemented a very simple CUDA version and then started to refine it, providing optimized versions to reduce transfer time and kernel time and an implementation that uses async and cuda streams to implement a pipeline.

5 Evidence of GCC auto-vectorization (used flags, what the compiler reported, short excerpt of the GCC report)

Flags used in Makefile to enable auto-vectorization: `OPTFLAGS = -O3 -march=native`. Key extract from the vectorization report output (flag: `VECFLAGS = -fopt-info-vec-all`):

```
src/partition.cpp:18:26: optimized: loop vectorized using 32 byte vectors
src/partition.cpp:18:26: optimized: loop vectorized using 16 byte vectors
src/partition.cpp:13:6: note: vectorized 1 loops in function.
```

The compiler successfully auto-vectorized the main loop using AVX2 instructions, as indicated by the GCC report ("loop vectorized using 32 byte vectors"). This corresponds to 256-bit SIMD execution, allowing the processing of four 64-bit elements per iteration. The second line tells us the fallback vectorization uses SSE (Streaming SIMD Extensions, older smaller vectors) where the register size is 16 bytes (128 bit). The third line tells us that the hash function is successfully inlined (the function call is replaced by the function body). Auto-vectorization was enabled by several key properties of the

kernel. First, the loop exhibits no data dependencies, as each iteration operates on independent elements. Second, the loop structure is simple and free of control flow, which facilitates compiler analysis. Third, the hash function consists only of arithmetic and bitwise operations, with no branches, making it highly suitable for SIMD execution. Additionally, memory accesses are sequential, ensuring good cache locality and efficient vector loads. A crucial design choice was replacing the modulo operation with a bitwise AND ($\& (P - 1)$), valid when the number of partitions is a power of two. This avoids expensive division operations and significantly improves vectorization efficiency, as bitwise operations are directly supported by SIMD instructions. Despite the theoretical speedup of up to $4\times$ (due to processing four 64-bit elements per SIMD instruction), the observed speedup is lower because the kernel is memory-bound (and also there is no full vectorization due to the limitation discussed above). Memory access latency and bandwidth limit the achievable performance, reducing the overall impact of SIMD acceleration.

6 Brief explanation of my intrinsic strategy

I start from the scalar execution, that processes 1 element per iteration, sequential. The goal is to transform it into the processing of 4 elements `uint64_t` at once using AVX2, 256-bit SIMD registers. My strategy involves replacing each scalar operation of the hash function with its SIMD equivalents (bitwise shifts `>>` were implemented using `_mm256_srli_epi64`, XOR operations `^` with `_mm256_xor_si256`, multiplication `*` with `_mm256_mullo_epi64` (it is not supported by the AVX2 so didn't work on the cluster and I had to change), and the final modulo operation was replaced by a bitwise AND `&` with `_mm256_and_si256`). The main loop loads four consecutive keys using `_mm256_loadu_si256` (safe even with unaligned memory), the hash computation is then applied in parallel to all elements within the SIMD register. However, my initial AVX2 implementation suffered from a bottleneck when converting 64-bit SIMD results to 32-bit outputs (the partition id is a 32-bit integer in my implementation), this part broke the SIMD flow by introducing a scalar loop and also adds memory overhead. In fact, a key challenge in the implementation is the lack of native support in AVX2 for packing 64-bit integers into 32-bit values. Since the output partition identifiers are 32-bit integers, a conversion step is required. This bottleneck is gradually minimized first using a combination of 128-bit operations and scalar extraction and then with only the use of fallback 128-bit vectorization for this very last small part. Nevertheless, the main computational part of the loop remains fully vectorized with AVX2, providing significant performance improvements. An alternative design choice would be to store partition identifiers as 64-bit integers instead of 32-bit, this would eliminate the conversion step in the AVX2 implementation at the cost of doubling the memory footprint of the output array, increasing memory bandwidth requirements. Since the kernel is memory-bound, increasing memory traffic would likely offset the benefits of improved vectorization introducing a trade-off.

Edit: I discovered that `_mm256_mullo_epi64` is available only in AVX512 so it is supported on my laptop but not on the cluster. So I replaced it with an efficient implementation, found on the web (reference in the source code) that split it into 32-bit multiplication that are vectorizable in AVX2 at the cost of introducing further overhead of operations/instructions and reducing efficiency. To speed up

one can simply remove the multiplication but it has strong bit mixing property so probably we lose good distribution property of the hash that I think is essential to split the keys in uniform set for the future of the project.

7 Experiments and optimization attempts

In the provided source code you can find two folders named `src` and `src2`. In `src` you can find the first implementation of scalar baseline, auto-vectorization and intrinsics. Then in `src2` you can find some experiments with some of the optimization techniques we've seen in class to improve performance like: **preallocating memory outside the hot loop**, **prefetching memory**, **loop unrolling**, **aligned memory allocation** and **aligned load/store**, trying to specify **more aggressive compilation flag** and also same other things that do not provide a real advantage like experimenting with some compiler hint (e.g. to inline the hash function) or working with pointer instead of `std::vector` for keys and `part_id`. In the following tables and results we refer to `novec` as the scalar baseline version, `vec` with the autovectorized version, `avx2` with the the intrinsics version. The versions with `opt` in their names refer to the "optimized" versions.

8 Performance results obtained on node09 and discussion of factor limiting performances(compute vs memory vs overhead)

Method	Exec Time (s)	Std	Speed-up (w.r.t. no_vec)	Throughput (keys/s)
novec	0.0238	0.0003	1.0000	705,912,578.1
novec_opt	0.0204	0.0003	1.1675	824,142,440.2
vec	0.0171	0.0001	1.3924	982,881,433.6
vec_opt	0.0172	0.0001	1.3784	973,057,607.3
avx2	0.0177	0.0001	1.3459	950,087,011.4
avx2_opt	0.0187	0.0001	1.2709	897,113,041.1
cuda_v1	0.3037	0.0118	0.0783	55,238,934.0
cuda_v2	0.0190	0.0004	1.2533	884,747,569.1
cuda_v3	0.0167	0.0003	1.4203	1,002,583,125.7

Table 1: Performance comparison of different vectorization and GPU methods. $N = 16\text{ M}$, $P = 1024$, compilation flag = OPTFLAG

In Table 1 you can see the result obtained on node09 (exclusive partition) for 2^{24} keys (16M), 1024 partitions. The tests have been performed using a simple script that run a certain number of times each version with the same parameters. For each versions (including gpu versions described below), you can see the execution time, expressed in s, that is the average of all execution times, the standard deviation, the speed-up computed with respect to the first simple scalar baseline and the throughput computed as the ratio between the total number of keys and the execution time. As we can see we the lowest execution time in this setting is reached by the auto-vectorized version that achieves a speed-up of near $1.4\times$ the scalar baseline. Also the manually vectorized version with intrinsics achieve a comparable performance. Another thing we observe is that the "optimized" versions with prefetching,

loop-unrolling, aligned memory...do not provide any sensible advantage, on the contrary a slight decrease in the performances of the vectorized versions. To perform a more robust test, one should test more isolately each factor to check their benefits. However, also from these tests we can see that our kernel is memory-bounded, as we thought the vectorization speedup is far from the 4x. This is due to the fact that the cpu work is very small, the hash is very simple and more, we do not vectorize it perfectly due to the bottleneck of the multiplication and the 64 $\rightarrow$ 32 bit conversion discussed above, that introduces a significant overhead of more operations/instructions.

9 Cuda optional part: correctness verification and breackdown of time components

For the cuda part you can find 3 different folders where you can find different experimental implementations. In the folder src you can find the first simple implementation, where we assign each thread to a different key. In practice, we instantiate enough thread to cover all the N keys, each thread performs a boundary check and if it is in the boundary it processes the key corresponding to its number. As we can see in the Table 1 this gpu baseline is very slow and inefficient even with respect to the cpu scalar baseline. The second version you can find in the folder cuda2 adopts two improvements: first it uses **pinned (page-locked) memory** and **grid-strided loop**. Pageable memory requires an additional staging step before GPU transfer, while pinned memory enables direct DMA transfer between host and device, significantly improving bandwidth and reducing latency. The grid-stride loop allows each thread to process multiple elements, improving scalability and ensuring full GPU utilization regardless of input size. It also decouples problem size from launch configuration. The performance bottleneck of the application is dominated by memory transfers host to device rather than kernel computation on device. By using pinned memory, we eliminate unnecessary staging copies and improve PCIe bandwidth. Additionally, the grid-stride loop ensures efficient utilization of GPU threads, enabling scalable and flexible execution independent of problem size. In cuda3 there is a different implementation: asynchronous memory transfers using CUDA streams to overlap data transfer and computation. The idea of the pipeline is to reduces idle time of both GPU and PCIe bus. For the cuda versions the transfer time is measured separately from the kernel time and the correctness check is implemented via the checksum.

Version	Exec Time (s)	Speed-up (vs V1)	Speed-up (vs previous)
CUDA V1	0.3013	1.0000	-
CUDA V2	0.0189	15.9047	15.9047
CUDA V3	0.0168	17.9729	1.1300

Table 2: Execution times and speed-ups of different CUDA versions. V1 is baseline, V2 is pinned memory + grid strided loop, v3 is asynch + cuda streams, N = 16M, P = 1024

CUDA Version	H2D (ms)	Kernel (ms)	D2H (ms)	Total CUDA Time (s)
CUDA V1	13.9099	1.3475	8.5688	0.3425
CUDA V2	11.0621	0.7738	6.0440	0.0191
CUDA V3	-	15.5423 (async)	-	0.0168

Table 3: Partial and total execution times of different CUDA versions. H2D: Host-to-Device transfer, D2H: Device-to-Host transfer. CUDA V3 show cuda asynch. time N = 16M, P = 1024

CUDA Version	H2D (ms)	Kernel (ms)	D2H (ms)	Total CUDA Time (s)
CUDA V1	1.5214	1.0507	1.0221	0.324962
CUDA V2	1.5445	0.8158	0.8456	0.004174
CUDA V3	-	3.1990 (async)	-	0.003999

Table 4: Partial and total execution times of different CUDA versions. N = 1M and P = 1024

Version	Exec Time (s)	Speed-up (vs V1)	Speed-up (vs previous)
Cuda V1	0.2464	1.0000	-
Cuda V2	0.003877	63.5698	63.5698
Cuda V3	0.003999	58.8370	0.9256

Table 5: Execution times and speed-ups of different CUDA versions. N = 1M, P = 1024

10 Comparison between different versions

In the following tables you can find comparisons between different versions varying the number of the keys or using more aggressive compilation flags.

Method	Exec Time (s)	Std	Speed-up (w.r.t. no_vec)	Throughput (keys/s)
novec	0.0236	0.0004	1.0000	711,903,490.9
novec_opt	0.0201	0.0003	1.1702	833,068,268.1
vec	0.0165	0.0001	1.4290	1,017,281,249.8
vec_opt	0.0165	0.0000	1.4294	1,017,567,537.2
avx2	0.0179	0.0000	1.3132	934,850,553.7
avx2_opt	0.0193	0.0000	1.2203	868,739,352.6

Table 6: Performance comparison of different vectorization methods with SUPEROPTIMIZATION (aka more aggressive) compilation flags. N = 16M, P = 1024

Method	Exec Time (s)	Std	Speed-up (w.r.t. no_vec)	Throughput (keys/s)
novec	0.0015	0.0000	1.0000	699,050,666.6
novec_opt	0.0013	0.0000	1.1440	806,596,923.1
vec	0.0011	0.0000	1.3558	953,250,909.1
vec_opt	0.0010	0.0000	1.4902	1,048,576,000
avx2	0.0012	0.0000	1.2861	873,813,333.3
avx2_opt	0.0012	0.0000	1.2658	873,813,333.3
cuda_v1	0.2499	0.0180	0.0060	4,195,982.4
cuda_v2	0.0039	0.0001	0.3815	268,865,641.0
cuda_v3	0.0040	0.0000	0.3704	262,144,000

Table 7: Performance comparison for N = 2^{20} elements using different vectorization and GPU methods. P = 1024.

Method	Exec Time (s)	Std (s)	Speed-up (vs novect)	Throughput (keys/s)
novect	0.3803	0.0065	1.0000	705,851,843.2
novect_opt	0.3364	0.0077	1.1303	797,965,089.1
vec	0.2721	0.0016	1.3978	986,532,363.1
vec_opt	0.2731	0.0026	1.3924	982,920,014.6
avx2	0.2960	0.0052	1.2848	906,876,540.5
avx2_opt	0.3192	0.0047	1.1913	840,963,208.0
cuda_v1	0.6719	0.0053	0.5660	399,516,975.7
cuda_v2	0.2857	0.0230	1.3309	986,532,363.1
cuda_v3	0.2310	0.0200	1.6461	1,162,058,251.0

Table 8: Performance results for $N = 2^{28}$ elements using different vectorization and CUDA methods. $P = 1024$

11 Suggested experiments

To see how good the hash is in providing a good distribution of the keys an histogram is computed and some statistical measure are printed. In particular the hash I implemented is very good in providing a near uniform distribution of the keys in all the partitions. In the following tables you can find small experiments of how different numbers of P impact on the performances.

Method	Exec Time (s)	Std (s)	Speed-up (vs novect)	Throughput (keys/s)
novect	0.0239	0.0004	1.0000	702,569,662.5
novect_opt	0.0205	0.0009	1.1662	819,349,827.4
vec	0.0166	0.0001	1.4407	1,012,198,228.3
vec_opt	0.0167	0.0005	1.4317	1,005,874,128.6
avx2	0.0180	0.0001	1.3232	929,672,100.4
avx2_opt	0.0195	0.0001	1.2251	860,752,976.0
cuda_v1	0.3029	0.0097	0.0788	55,389,763.7
cuda_v2	0.0190	0.0003	1.2568	883,016,713.0
cuda_v3	0.0168	0.0003	1.4242	1,000,617,050.2

Table 9: Performance comparison of scalar, vectorized, and CUDA implementations. $N = 16$ M, $P = 512$

Method	Exec Time (s)	Std (s)	Speed-up (vs novect)	Throughput (keys/s)
novect	0.0238	0.0003	1.0000	704,910,825.3
novect_opt	0.0202	0.0004	1.1774	829,943,061.8
vec	0.0166	0.0001	1.4371	1,013,028,207.1
vec_opt	0.0165	0.0001	1.4399	1,014,990,902.6
avx2	0.0184	0.0016	1.2914	910,335,322.7
avx2_opt	0.0194	0.0001	1.2237	862,583,351.7
cuda_v1	0.3033	0.0110	0.0785	55,316,074.4
cuda_v2	0.0189	0.0004	1.2618	889,429,774.5
cuda_v3	0.0167	0.0003	1.4211	1,001,752,821.0

Table 10: Performance comparison of vectorization and CUDA methods. $N = 16$ M, $P = 256$

References

- [1] Murmurhash. <https://en.wikipedia.org/wiki/MurmurHash>, 2025.

- [2] David Stafford. Better bit mixing - improving on murmurhash3's finalizer. <https://zimbry.blogspot.com/2011/09/better-bit-mixing-improving-on.html>. Blog post.